



CARRERA DE ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL

MEMORIA DEL TRABAJO FINAL

Desarrollo de un *pipeline* de aprendizaje continuo para chatbots basados en PLN

Autor:

Ing. Porra Bustos, Matias Exequiel (UTN-FRLR)

Director:

Dr. Ing. Cárdenas Rodrigo (FIUBA)

Jurados:

Esp. Ing. Torrent Leandro (FIUBA)

Nombre del jurado 2 (pertenencia)

Nombre del jurado 3 (pertenencia)

*Este trabajo fue realizado en la Ciudad Autónoma de Buenos Aires,
entre mayo de 2024 y junio de 2025.*

Resumen

El presente trabajo desarrolla una solución avanzada para la automatización de la comunicación empresarial, orientada a emprendedores y pequeñas empresas.

El sistema desarrollado consiste en un chatbot que integra técnicas de procesamiento del lenguaje natural (PLN) y gestión de bases de datos, articulado en un pipeline de aprendizaje continuo. Este chatbot está diseñado para optimizar la interacción con los clientes, ya que genera respuestas precisas y relevantes a sus consultas en lenguaje natural. Además, el sistema se retroalimenta de las interacciones previas para mejorar su rendimiento de manera constante y adaptarse mejor a las necesidades de los usuarios.

Agradecimientos

A mi novia por apoyo incondicional durante todo el proceso de desarrollo de este trabajo.

A mi familia, por estar siempre presente.

A mi tutor, por su guía y su conocimiento.

A Michael Schreiber por sus grandes aportes y guía en mi formación profesional en IA.

Índice general

Resumen	I
1. Introducción general	1
1.1. Motivación	1
1.1.1. ¿Qué es un chatbot?	1
1.1.2. ¿Por qué un chatbot?	1
1.2. Alcance y objetivos	2
1.2.1. Objetivos principales	2
1.2.2. Alcance del trabajo	3
1.3. Estado del arte	3
1.3.1. Recursos para chatbots	3
Servicios basados en APIs	3
Modelos locales	4
1.3.2. Tendencias actuales	4
2. Introducción específica	5
2.1. Requerimientos	5
2.2. Modelos de inteligencia artificial para PLN	6
2.2.1. Modelos de Lenguaje modernos	7
2.3. Modelos de chatbots: RAG vs Fine-Tuning	7
2.3.1. Retrieval Augmented Generation (RAG)	7
2.3.2. Fine-Tuning	8
2.3.3. Ventajas y desventajas de los modelos RAG y Fine-Tuning	9
3. Diseño e implementación	11
3.1. Diseño de la arquitectura	11
3.1.1. Propuestas de implementación	11
3.1.2. Problemas y mitigaciones	14
3.1.3. Elección de la arquitectura de chatbot	16
3.2. Arquitectura final propuesta	17
3.2.1. Tecnologías suplementarias	18
3.2.2. Funcionamiento general	18
3.3. Implementación	20
3.3.1. Preparación de los datos de contexto	20
3.3.2. Sistema de reentrenamiento con historiales y evaluación automatizada de interacciones	21
3.4. Despliegue de la interfaz de pruebas	24
4. Ensayos y resultados	27
4.1. Evaluación de escenarios del chatbot	27
4.1.1. Escenario 1: Interacción básica	27
4.1.2. Escenario 2: Consultas con múltiples pasos	27
4.1.3. Escenario 3: Consultas fuera de contexto	28

4.2.	Resultados de las pruebas	28
4.2.1.	Pruebas para la version 0	29
	Pruebas de interacción básica	29
	Pruebas de consultas con múltiples pasos	29
	Pruebas de consultas fuera de contexto	29
	Conclusiones de la version 0	29
4.2.2.	Pruebas para la version 1	29
	Pruebas de interacción básica	29
	Pruebas de consultas con múltiples pasos	29
	Pruebas de consultas fuera de contexto	29
	Conclusiones de la version 1	29
4.2.3.	Pruebas para la version 2	29
	Pruebas de interacción básica	29
	Pruebas de consultas con múltiples pasos	29
	Pruebas de consultas fuera de contexto	29
	Conclusiones de la version 2	29
4.2.4.	Pruebas para la version 3	29
	Pruebas de interacción básica	29
	Pruebas de consultas con múltiples pasos	29
	Pruebas de consultas fuera de contexto	29
	Conclusiones de la version 3	29
4.3.	Mejoras y optimizaciones propuestas	29
4.3.1.	Mejoras en la gestión del contexto	29
4.3.2.	Optimización del modelo de lenguaje	29
4.3.3.	Integración con sistemas externos	29
4.3.4.	Mejoras en la interfaz de usuario	29

Bibliografía

Índice de figuras

1.1. Relación entre los objetivos centrales.	2
2.1. Diagrama de flujo de una consulta en un chatbot con arquitectura RAG.	8
3.1. Diagrama general del sistema para la versión 1.	12
3.2. Diagrama general del sistema para la versión 2.	13
3.3. Ejemplo de redefinición de pregunta.	15
3.4. Frontend del chatbot.	17
3.5. Diagrama general del flujo del pipeline del sistema.	18
3.6. Diagrama de flujo de funcionamiento.	19
3.7. Diagrama de flujo de observación de métricas.	22
3.8. Métricas por categoría.	22
3.9. Ejemplo de frontend que muestra métricas para preguntas buenas y malas.	23
3.10. Frontend creado para la administración de contextos.	25

Índice de tablas

2.1. Ventajas y desventajas de los modelos RAG y Fine-Tuning.	9
---	---

Capítulo 1

Introducción general

En este capítulo se presentan las motivaciones que impulsaron el desarrollo del sistema, diseñado para automatizar la comunicación entre emprendedores y sus clientes. Se ofrece, además, una explicación detallada sobre qué son los chatbots y las razones por las que su uso resulta fundamental en este contexto. Finalmente, se describen los objetivos principales del trabajo, centrados en la facilidad de implementación y en la mejora de la eficiencia operativa, junto con el alcance definido durante la planificación.

1.1. Motivación

El sistema desarrollado en el presente trabajo surge de la necesidad de proporcionar a emprendedores una herramienta que les permita automatizar la comunicación con sus clientes de manera eficiente y personalizada. Esta herramienta les ofrece una ventaja competitiva al optimizar la gestión de sus interacciones con los usuarios y les permite mejorar la eficiencia operativa, además de reducir los costos asociados a la atención al cliente.

1.1.1. ¿Qué es un chatbot?

Un chatbot es un programa de software que emplea técnicas de Procesamiento del Lenguaje Natural (PLN) [1], para interpretar y contestar automáticamente las consultas de los usuarios de manera coherente y eficiente. Esto facilita la automatización de interacciones comunes y mejora la experiencia del cliente.

1.1.2. ¿Por qué un chatbot?

A continuación, se destacan algunas de las razones principales por las que un chatbot es la solución ideal para este trabajo:

- Disponibilidad 24/7: los chatbots están disponibles en todo momento, lo que les permite a las empresas atender consultas de sus clientes a cualquier hora del día.
- Reducción de costos: al automatizar la atención al cliente, los chatbots ayudan a las empresas a reducir los costos asociados al personal humano sin comprometer la calidad del servicio.
- Recolección de datos valiosos: los chatbots pueden recopilar y analizar información relevante sobre las interacciones con los clientes, lo que les permite a las empresas ajustar sus estrategias de manera eficiente.

- Optimización de tiempos de respuesta: la capacidad de generar respuestas inmediatas en función de consultas predefinidas optimiza los tiempos de respuesta.

1.2. Alcance y objetivos

1.2.1. Objetivos principales

El trabajo se enfoca en tres objetivos principales que se interrelacionan para ofrecer una solución integral a pequeños emprendedores. Cada uno de estos objetivos está orientado a asegurar que la herramienta sea accesible, fácilmente implementable y de bajo costo, para garantizar una experiencia eficiente y optimizada para las empresas que la adopten.

A continuación, se detallan los objetivos principales:

- Proporcionar acceso a pequeños emprendedores: desarrollar un sistema que sea de fácil adopción y pueda ser utilizado por pequeñas empresas, independientemente de sus conocimientos técnicos.
- Garantizar una fácil implementación: diseñar una solución replicable, de modo que cualquier empresa pueda contar con el chatbot simplemente al proporcionar el contexto necesario para su entrenamiento. No será necesario disponer de infraestructura compleja ni personal especializado.
- Reducir el costo de mantenimiento: proporcionar una herramienta con un bajo costo de mantenimiento, tanto en términos económicos como de tiempo, para que los emprendedores puedan centrarse en su negocio principal.

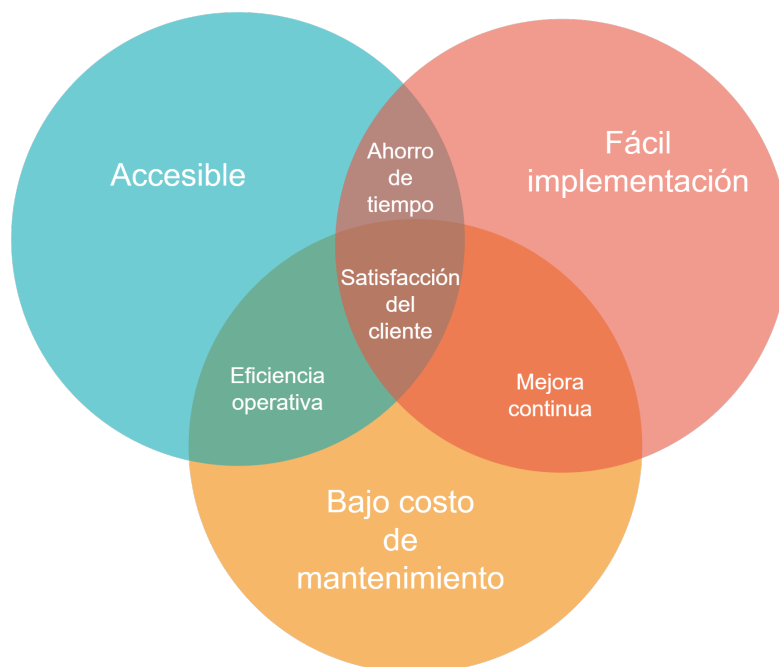


FIGURA 1.1. Relación entre los objetivos centrales.

1.2.2. Alcance del trabajo

Durante la planificación del trabajo se propusieron las siguientes actividades:

- Diseño, desarrollo e implementación de un *pipeline* completo para la creación y gestión de chatbots basados en PLN, con capacidad de aprendizaje continuo.
- Desarrollo de algoritmos para el preprocesamiento de información y generación de respuestas relevantes y coherentes.
- Establecimiento de métricas y procedimientos de evaluación para medir la calidad y eficacia de las respuestas del chatbot.
- Implementación de un sistema de retroalimentación que utilice los datos de interacciones con usuarios para mejorar el rendimiento del modelo.

El presente trabajo excluye:

- Desarrollo de interfaces de usuario avanzadas, por fuera de las básicas necesarias para probar el pipeline.
- Integración con sistemas externos, como bases de datos o plataformas de gestión de clientes.
- Implementación del sistema en un entorno productivo.
- Actividades de marketing o promoción del producto final.

1.3. Estado del arte

Los chatbots se han integrado en numerosos aspectos de la vida diaria y en diversos canales de comunicación. Se encuentran presentes en redes sociales como Facebook e Instagram, donde facilitan la interacción con empresas. Además, muchas organizaciones y entidades gubernamentales, han implementado chatbots para mejorar la atención al ciudadano. Plataformas como IBM Watson, Google Dialogflow y Microsoft Bot Framework permiten a las empresas crear chatbots personalizados con gran flexibilidad.

1.3.1. Recursos para chatbots

Servicios basados en APIs

Los servicios basados en API¹ simplifican la implementación de chatbots al ofrecer interfaces estandarizadas para acceder a funcionalidades avanzadas. Estos servicios permiten una mayor personalización y flexibilidad en el diseño de chatbots, que facilitan la integración con otros sistemas y plataformas. La utilización de APIs permite a las empresas adaptar las soluciones a sus necesidades específicas sin necesidad de desarrollar toda la infraestructura desde cero.

¹API (Interfaz de Programación de Aplicaciones) es un conjunto de reglas y protocolos que permite que diferentes aplicaciones se comuniquen entre sí.

Modelos locales

Los modelos locales como Llama 3.1, Phi 3, Gemma y Mistral son soluciones robustas que operan sin depender de servicios externos. Cada uno de estos ofrece características y ventajas específicas, como mayor control sobre los datos y la posibilidad de operar en entornos con restricciones de privacidad. Su uso es particularmente valioso cuando se requiere un alto nivel de personalización y seguridad en la gestión de datos.

1.3.2. Tendencias actuales

Cada vez más empresas optan por utilizar servicios basados en API, que han hecho más accesible y económico el despliegue de chatbots sofisticados. Estas herramientas simplifican la implementación y permiten la creación rápida de soluciones adaptadas a las necesidades específicas de cada empresa o entidad. Aunque los modelos locales son valiosos por su robustez y capacidad para operar sin depender de servicios externos, son útiles especialmente cuando se requiere un mayor control o confidencialidad.

Capítulo 2

Introducción específica

Este capítulo explora los requisitos fundamentales para el desarrollo de un sistema de chatbot, que abarcan aspectos funcionales, de documentación, pruebas, interfaz y rendimiento. Además, revisa los diferentes tipos de chatbots y modelos de inteligencia artificial, con énfasis en sus características y aplicaciones en el procesamiento de lenguaje natural.

2.1. Requerimientos

En esta sección se presentan los requerimientos específicos de software del sistema. Estos se tuvieron en cuenta al inicio del desarrollo de este proyecto, cuyo fin es construir un chatbot inteligente basado en arquitectura RAG, optimizado para brindar respuestas contextualizadas, precisas y eficientes a través de una interfaz adaptable y trazabilidad completa de interacciones.

1. Requerimientos funcionales fueron:

- a) El sistema debe ser capaz de procesar y comprender mensajes de texto entrantes.
- b) El chatbot debe poder proporcionar respuestas relevantes y precisas a las consultas de los usuarios.
- c) Los usuarios deben tener la posibilidad de interactuar con el chatbot a través de una interfaz de usuario.
- d) Se requiere una interfaz de usuario que facilite el proceso de reentrenamiento del chatbot mediante el uso de los historiales de conversación.

2. Requerimientos de documentación:

- a) Documentar detalladamente las tecnologías utilizadas y sus principales características, así como las particularidades de diseño de cada una.
- b) El sistema no almacenará datos personales de los usuarios, sino que se limitará a responder preguntas frecuentes. En caso de requerirse almacenamiento de datos, se utilizará una plataforma con medidas de seguridad integradas, como autenticación mediante logins con Google.
- c) Entregar una memoria técnica que contenga información detallada sobre la implementación del sistema, que incluya aspectos técnicos, arquitectura y decisiones de diseño.

- d) Proporcionar un registro de avance que documente los hitos alcanzados durante el desarrollo del proyecto, que contemple fechas de cumplimiento y descripciones de las tareas realizadas.
3. Requerimientos de Testing:
 - a) Se llevarán a cabo pruebas en diferentes escenarios y con diferentes tipos de contexto de datos, para garantizar la fiabilidad y precisión del chatbot.
 4. Requerimientos de la Interfaz:
 - a) Se requiere una interfaz interactiva, que permita hacer preguntas y obtener respuestas en tiempo real dentro del mismo entorno de interacción.
 5. Requerimientos de Rendimiento:
 - a) Se establece un límite máximo de tiempo de respuesta de 1 minuto, para asegurar una experiencia satisfactoria para el usuario.

2.2. Modelos de inteligencia artificial para PLN

El procesamiento de lenguaje natural (PLN) ha avanzado significativamente gracias al desarrollo de diversas técnicas de inteligencia artificial. Estas se pueden clasificar en varias categorías, cada una con su propio enfoque y aplicación.

Modelos basados en reglas: estos modelos utilizan gramáticas y diccionarios para analizar el lenguaje. Aunque son efectivos en tareas específicas, su rigidez y la necesidad de una extensa programación manual limitan su aplicabilidad en contextos más amplios.

Modelos estadísticos: a medida que los datos comenzaron a acumularse, los modelos estadísticos, como los n-gramas, se convirtieron en populares. Estos modelos predicen la probabilidad de una palabra en función de las anteriores. Sin embargo, su dependencia de datos limitados puede resultar en un contexto insuficiente, lo que afecta la calidad de las predicciones.

Modelos de aprendizaje profundo: con el auge del aprendizaje profundo, arquitecturas como RNN (Redes Neuronales Recurrentes), LSTM (Memoria a Largo Plazo) y transformers han revolucionado el PLN. Estos modelos pueden captar patrones complejos en grandes volúmenes de datos, lo que les permite realizar tareas como la traducción automática y la generación de texto de manera más efectiva.

2.2.1. Modelos de Lenguaje modernos

Modelos como BERT (Bidirectional Encoder Representations from Transformers) y GPT (Generative Pre-trained Transformer) han transformado el campo del procesamiento de lenguaje natural (PLN), al ofrecer enfoques innovadores para comprender y generar texto.

- **GPT:** este modelo utiliza técnicas de aprendizaje profundo para generar texto coherente y contextualmente relevante. Su capacidad para crear respuestas naturales ha revolucionado la interacción de los chatbots con los usuarios, lo que permite conversaciones más fluidas y precisas.
- **BERT:** a diferencia de otros modelos, BERT se centra en el análisis bidireccional del texto, lo que le permite entender el contexto de manera más profunda. Esta característica mejora la identificación de intenciones y la respuesta a preguntas complejas, lo que ayuda a crear chatbots más competentes en diálogos matizados.

2.3. Modelos de chatbots: RAG vs Fine-Tuning

Los chatbots se pueden clasificar según sus arquitecturas y métodos de entrenamiento. Entre las más destacadas se encuentran Retrieval Augmented Generation (RAG) y *fine-tuning*. Ambas arquitecturas ofrecen ventajas complementarias, lo que las convierten en opciones populares en el desarrollo de chatbots efectivos [2].

2.3.1. Retrieval Augmented Generation (RAG)

RAG integra técnicas de generación de lenguaje natural con mecanismos de recuperación de información que operan sobre una base de conocimiento estructurada. Ante una consulta del usuario, el modelo emplea *embeddings*¹ para localizar y extraer los segmentos más relevantes del contexto almacenado, con el fin de generar una respuesta coherente y fundamentada. Este conocimiento relevante se inyecta en el *prompt*² enviado al modelo de lenguaje (LLM). El *prompt* incluye tanto la pregunta como el contexto relacionado, lo que permite a la LLM generar respuestas más precisas y contextualizadas. La incorporación de datos externos en este proceso enriquece el conocimiento del modelo en tiempo real y disminuye las posibilidades de alucinaciones—respuestas que, aunque parecen plausibles, son incorrectas. Además, RAG utiliza bases de datos vectoriales y la búsqueda semántica para recuperar información relevante, lo que mejora la precisión y relevancia de las respuestas del chatbot. Este enfoque es especialmente útil en situaciones donde se requiere información actualizada o especializada, como en consultas de productos o en el soporte técnico.

¹Embeddings [representaciones vectoriales].

²prompt [entrada o solicitud al modelo de lenguaje].

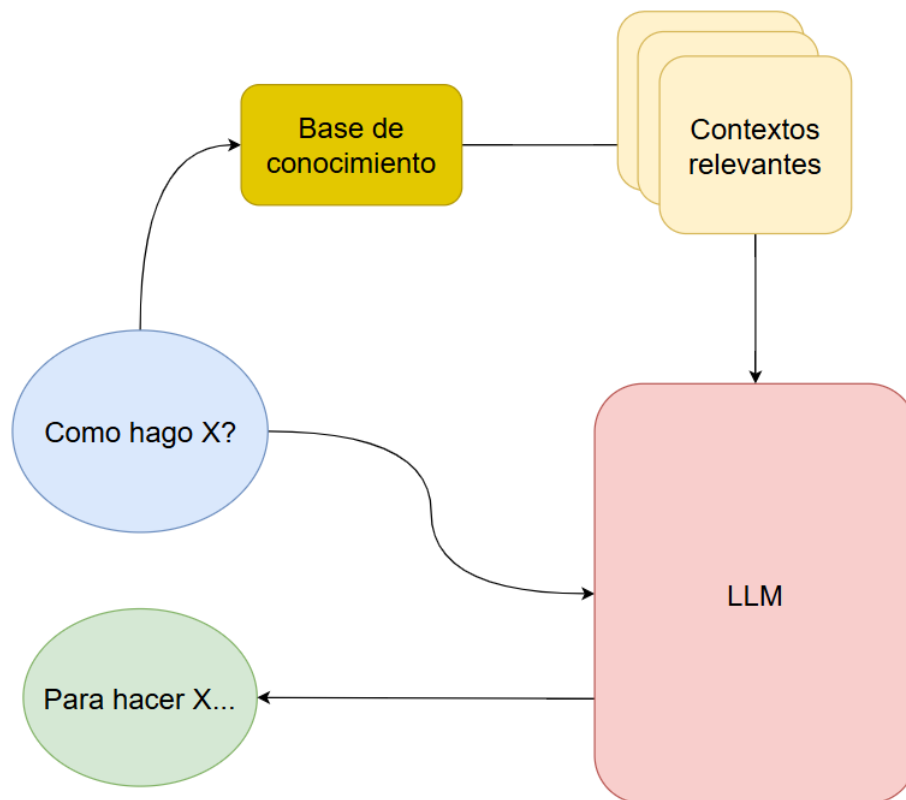


FIGURA 2.1. Diagrama de flujo de una consulta en un chatbot con arquitectura RAG.

2.3.2. Fine-Tuning

El *fine-tuning* implica ajustar un modelo de lenguaje preentrenado en un conjunto de datos específico para mejorar su comportamiento en contextos particulares. Este proceso optimiza la efectividad del modelo en aplicaciones industriales específicas, como en medicina, derecho o ingeniería. Sin embargo, una desventaja del *fine-tuning* es que el modelo resultante puede quedar congelado en un estado particular, lo que limita su adaptabilidad a nuevos contextos sin un nuevo ciclo de entrenamiento. Aunque el *fine-tuning* proporciona respuestas más precisas en contextos específicos, puede carecer de la flexibilidad de RAG, que permite incorporar datos contextuales en tiempo real.

2.3.3. Ventajas y desventajas de los modelos RAG y Fine-Tuning

A continuación, se presenta una tabla que compara las principales ventajas y desventajas de los modelos RAG (Retrieval-Augmented Generation) y *Fine-Tuning*. Esta comparación proporciona una visión clara de las características distintivas de cada enfoque, lo que permite una evaluación informada de su idoneidad en función de los requisitos específicos del entorno de implementación.

TABLA 2.1. Ventajas y desventajas de los modelos RAG y Fine-Tuning.

Modelo	Ventaja	Desventaja
RAG	■ Permite incorporar información contextual actualizada en tiempo real. ■ Reduce la probabilidad de alucinaciones al usar datos relevantes. ■ Mejora la precisión y relevancia de las respuestas del chatbot.	■ Requiere una infraestructura adecuada para la recuperación de información. ■ Dependencia de la calidad y disponibilidad de la base de datos. ■ Puede ser más complejo de implementar y mantener.
Fine-Tuning	■ Mejora la calidad de las respuestas al adaptar el modelo a un dominio específico. ■ Proporciona un rendimiento más consistente en contextos bien definidos. ■ Permite la optimización de la latencia al trabajar con un modelo entrenado.	■ El modelo se congela en el tiempo y no se adapta a cambios contextuales. ■ Requiere un conjunto de datos específico para el entrenamiento, lo que puede ser costoso. ■ Menor flexibilidad para abordar consultas inesperadas o no entrenadas.

Capítulo 3

Diseño e implementación

3.1. Diseño de la arquitectura

3.1.1. Propuestas de implementación

Durante el desarrollo del sistema se elaboraron cuatro versiones principales, cada una representa un cambio significativo en la tecnología, la arquitectura o el enfoque funcional del chatbot. A continuación, se describen dichas iteraciones:

Versión 0 - RAG básico con Pinecone

La necesidad inicial consistía en construir un prototipo funcional para validar la arquitectura RAG mediante herramientas accesibles.

Descripción: se implementó una arquitectura RAG elemental que utiliza una LLM (GPT-3.5-turbo) y el servicio de vectorstore¹ Pinecone. El flujo básico consistía en preprocesar documentos, generar embeddings, almacenarlos en Pinecone y recuperar los fragmentos relevantes a partir de la pregunta del usuario, con el objetivo de construir un prompt enriquecido que sería enviado a la LLM.

Problemas detectados: aunque Pinecone ofrecía un buen desempeño en la recuperación de información, su costo por consulta representaba un obstáculo considerable para su adopción en entornos de producción. Además, la trazabilidad del sistema era limitada, ya que no existían mecanismos para evaluar la calidad de los fragmentos obtenidos ni auditar el proceso de recuperación.

Lo anterior evidenció la necesidad de un sistema de almacenamiento local que permitiera un mayor control sobre la generación, almacenamiento y recuperación de los embeddings. Asimismo, se requerían herramientas que facilitaran tanto la evaluación de las respuestas como la auditoría del flujo completo de recuperación de contextos relevantes.

¹Vectorstore (almacenamiento de vectores) es una estructura de datos especializada en almacenar y recuperar representaciones vectoriales de documentos o fragmentos de texto. En el contexto de RAG, el vectorstore almacena los vectores de contexto generados a partir de documentos que han sido procesados y transformados en embeddings. Estos vectores permiten realizar búsquedas eficientes y precisas, lo que ayuda al modelo a recuperar fragmentos relevantes para generar respuestas más contextualizadas sin tener que reentrenar el modelo de lenguaje.

Versión 1 - Integración de DSPy, migración a LanceDB y uso de NemoGuardrails

Razón del cambio: mejorar la calidad de las respuestas, reducir los costos de infraestructura y establecer una capa de control conversacional previa a la ejecución del pipeline RAG.

Descripción: en esta versión se reemplazó Pinecone por LanceDB, una base de datos de vectores que permite almacenamiento local y proporciona mayor control sobre la estructura y el acceso a los embeddings, lo que elimina los costos asociados a servicios externos.

Se integró DSPy, un framework que permite definir prompts de manera modular y declarativa, lo que facilitó una implementación más mantenible y escalable de la arquitectura RAG.

Asimismo, se añadió una capa de filtrado con NemoGuardrails² para bloquear preguntas fuera de dominio, junto con un sistema de evaluación automática que puntuaba las respuestas de la LLM, y permitía monitorear su rendimiento. Estos componentes fueron implementados como módulos independientes en DSPy, lo que mejoró la modularidad y claridad del sistema.

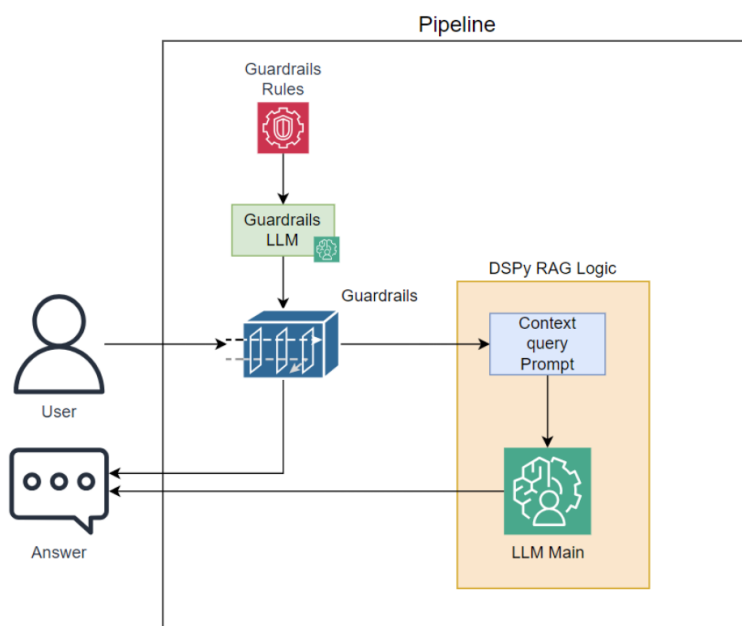


FIGURA 3.1. Diagrama general del sistema para la versión 1.

Problemas detectados: aunque se logró una mayor robustez y control, surgieron diversos desafíos. NemoGuardrails presentó un rendimiento limitado en español y una baja capacidad de personalización, debido a su fuerte orientación hacia el idioma inglés, lo que redujo la eficacia del filtrado y motivó la búsqueda de alternativas más flexibles.

²NeMo Guardrails es un sistema de control diseñado para filtrar y gestionar las interacciones con modelos de lenguaje, de manera que las respuestas sean apropiadas y alineadas con las políticas predefinidas. En el contexto de RAG, se utiliza para clasificar y redirigir consultas, lo que garantiza que el sistema mantenga un comportamiento controlado y seguro.

Además, si bien se incorporó un sistema de evaluación automática, aún no se contaba con trazabilidad completa de las interacciones ni con segmentación por métricas, aspectos que se abordarían en versiones posteriores.

Versión 2 - Guardrails personalizados, almacenamiento estructurado de chats e implementación del frontend y backend de pruebas

Razón del cambio: necesidad de una mayor precisión en el filtrado y una mejor trazabilidad de las conversaciones.

Descripción: se reemplazó NemoGuardrails por una implementación propia, basada en LLMs livianos y reglas personalizadas escritas en código con prompts específicos. Esto permitió una mayor flexibilidad y una mejor adaptación al idioma español.

Además, se comenzó a almacenar localmente la conversación completa de cada usuario, con preguntas, respuestas y el contexto previo, en la memoria RAM³. Esta funcionalidad dotó al sistema de memoria conversacional, lo que mejoró la coherencia de las respuestas y habilitó análisis posteriores sobre el historial de interacción.

En paralelo, todas las interacciones del asistente (preguntas, respuestas, errores y rechazos por guardrails) comenzaron a registrarse en una base de datos NoSQL, lo que facilitó la trazabilidad de cada interacción y el análisis detallado del comportamiento conversacional.

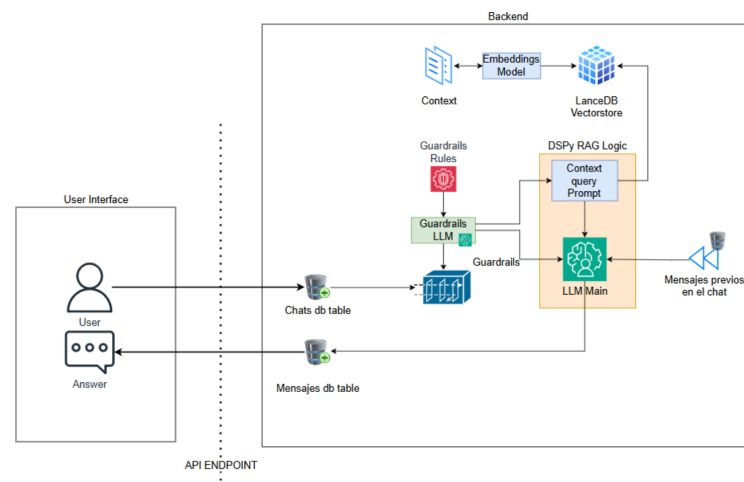


FIGURA 3.2. Diagrama general del sistema para la versión 2.

Problemas detectados: aunque el nuevo sistema de guardrails ofreció un mayor control, la forma en que se persistían los historiales de conversación generó un aumento significativo en el uso de memoria RAM, lo que limitó la escalabilidad del sistema. Además, la latencia de respuesta aumentó considerablemente y afectó la experiencia del usuario.

³RAM (Memoria de Acceso Aleatorio, por sus siglas en inglés) es un tipo de memoria volátil utilizada por los sistemas informáticos para almacenar datos de forma temporal y rápida. En el contexto de los modelos de lenguaje, RAM es crucial para mantener los datos y contextos relevantes durante la ejecución de tareas de procesamiento, y permite un acceso rápido a la información mientras el modelo genera respuestas.

Por último, si bien el almacenamiento de las conversaciones facilitó la gestión del contexto en interacciones encadenadas, se identificó que, en casos donde la nueva pregunta dependía de la anterior, el contexto no era interpretado correctamente. Por ejemplo:

- Usuario: "¿Tienen stock del producto X?"
- Robot: "Sí, tenemos stock del producto X."
- Usuario: "¿Cuánto cuesta?"
- Robot: "¿Podrías aclarar de qué producto hablas?"

En este caso, el sistema no logró identificar que la pregunta hacía referencia al producto mencionado previamente. Esta limitación evidenció la necesidad de incorporar un sistema de reformulación de preguntas que permitiera enriquecer el contexto conversacional mediante el uso de palabras clave que referencien el historial de la interacción.

Versión 3 - Integración de Langfuse, clasificación y evaluación automática

Razón del cambio: necesidad de un sistema robusto para trazabilidad, evaluación y mejora continua.

Descripción: se integró Langfuse como sistema observador no intrusivo para registrar todas las interacciones del sistema. Además, se añadieron dos nuevas tareas en la capa de guardrails: la primera, redefinir preguntas, que genera una nueva oración con palabras clave para buscar los contextos en la vectorstore; la segunda, clasificar consultas, que organiza las preguntas según su temática, como por ejemplo: Productos, Precios, Delivery, Queja, Saludo, entre otras. También se eliminaron bases de datos auxiliares y se centralizó la gestión de la trazabilidad en Langfuse, que ahora se encarga de registrar todos los eventos: consultas, respuestas, errores, rechazos por guardrails, consumo de tokens, costos, latencia, etc.

Problemas detectados: algunos temas, como las quejas, mostraron un rendimiento inferior, lo que sugiere la necesidad de expandir y refinar el contexto disponible en versiones futuras.

3.1.2. Problemas y mitigaciones

Durante las primeras iteraciones surgieron limitaciones clave: costos por servicios externos (Pinecone, API de LLMs), pobre trazabilidad del sistema y ausencia de control sobre la evaluación de calidad. Estos problemas fueron mitigados mediante las siguientes acciones:

- **Migración a LanceDB para almacenamiento local sin costo.** LanceDB es una base de datos vectorial optimizada para uso local. Su adopción permitió eliminar la dependencia de servicios como Pinecone, que implicaban costos por cada consulta. Esta decisión fue especialmente útil durante la etapa de desarrollo y pruebas, ya que redujo los costos operativos. Sin embargo, se observó que LanceDB puede presentar latencias más altas en comparación con soluciones SaaS (Software as a Service), lo cual debe considerarse en entornos de producción. Por otro lado, para aplicaciones a gran escala, su



FIGURA 3.3. Ejemplo de redefinición de pregunta.

uso local podría representar un ahorro sustancial al evitar tarifas por tokens o el uso de APIs externas.

- **Uso de DSPy para estructurar la lógica RAG y mejorar la modularidad.** DSPy es una biblioteca que permite construir y optimizar pipelines de IA mediante programación declarativa. Se utilizó para implementar de manera modular el sistema RAG, y se definieron claramente las etapas de recuperación y generación. Esta implementación facilitó la trazabilidad del proceso completo, y permitió auditar tanto la recuperación de contextos como la generación de respuestas. Además, su enfoque declarativo simplificó el desarrollo, mejoró la mantenibilidad y facilitó la introducción de mejoras incrementales al sistema.
- **Reemplazo de NemoGuardrails por guardrails personalizados.** NemoGuardrails no ofrecía resultados adecuados para preguntas en español. Por esta razón, se diseñó una solución propia basada en prompts simples con modelos LLM. Esta implementación personalizada fue capaz de detectar y filtrar preguntas sobre política, religión y otros temas sensibles. Además, se utilizó como etapa inicial para la clasificación temática y la redefinición de preguntas en etapas posteriores.
- **Incorporación de Langfuse como observador centralizado y sistema de métricas.** Langfuse es una herramienta de observabilidad que permite registrar, analizar y etiquetar cada interacción con el modelo. Su integración aportó trazabilidad completa al sistema, facilitó el monitoreo de métricas clave (como latencia, consumo de tokens, respuestas generadas) y habilitó

la evaluación automática mediante la implementación de agentes de evaluación con IA en su plataforma. Gracias a esta integración, se logró una visión integral del rendimiento del sistema, lo que permitió identificar áreas de mejora y optimizar el flujo de trabajo. Esta capacidad eliminó la etapa de DSPy que se encargaba de la evaluación automática posterior a la respuesta de la LLM. Esto redujo la latencia del sistema y permitió una mejor trazabilidad de las interacciones, ya que Langfuse almacena todos los eventos generados por el sistema, con la evaluación automática. Esto facilita la auditoría del funcionamiento del sistema y mejora el contexto disponible en la etapa de análisis de métricas.

3.1.3. Elección de la arquitectura de chatbot

La arquitectura final del chatbot se basa en el enfoque RAG (Retrieval-Augmented Generation), una técnica que ha demostrado ser altamente efectiva para responder preguntas contextualizadas sin necesidad de realizar un reentrenamiento completo del modelo de lenguaje (LLM). Esta decisión se tomó para ofrecer un sistema flexible, escalable y de bajo costo operativo, especialmente orientado a pequeñas empresas y emprendedores que requieren soluciones ágiles y sostenibles.

El uso de RAG permite mantener el modelo LLM independiente de la base de conocimiento, lo que implica que se pueden incorporar nuevos documentos, actualizaciones o correcciones al modificar los datos almacenados en el vectorstore, sin necesidad de ajustar los parámetros del modelo. Esta separación entre el modelo y la información facilita enormemente las tareas de mantenimiento y mejora continua del sistema.

La elección de esta arquitectura RAG se fundamentó en su capacidad para ofrecer un equilibrio óptimo entre precisión, costo y mantenibilidad. Durante el desarrollo del proyecto, OpenAI lanzó GPT-4o mini, un modelo que presentó un excelente balance entre rendimiento y economía, con capacidades superiores a las alternativas locales disponibles en ese momento. Aunque se evaluó la posibilidad de implementar un modelo local con Ollama⁴, la combinación de GPT-4o mini con LanceDB resultó ser más eficiente en términos de costo-beneficio, ya que eliminaba la necesidad de infraestructura adicional para hospedar y mantener modelos locales. Esta decisión redujo significativamente el costo por consulta, y mantuvo un alto estándar de calidad en las respuestas, lo que hizo que la solución fuera más accesible para pequeñas empresas y emprendedores. Además, el uso de componentes modulares como DSPy facilitó la adaptación del sistema a nuevos modelos o proveedores de LLMs en el futuro, sin necesidad de rediseñar toda la arquitectura.

⁴Ollama es una plataforma que permite ejecutar modelos de lenguaje de manera local, y ofrece una alternativa a los servicios en la nube. En el contexto de RAG, Ollama permite el uso de modelos de lenguaje sin necesidad de depender de APIs externas, lo que puede ofrecer mayor control sobre los datos y reducir costos operativos. Ollama facilita la integración de modelos LLM en entornos locales, y permite realizar consultas y generar respuestas con los contextos recuperados de un vectorstore, lo cual es clave para la arquitectura de Retrieval-Augmented Generation.

3.2. Arquitectura final propuesta

La arquitectura final del sistema está diseñada para ofrecer una solución robusta, eficiente y fácilmente escalable para la automatización de respuestas contextuales. Está compuesta por los siguientes bloques funcionales:

- **Frontend:** interfaz renovada para usuarios y administradores. Permite enviar preguntas al sistema, visualizar respuestas, acceder a métricas y gestionar los contextos mediante un formulario ilustrativo.

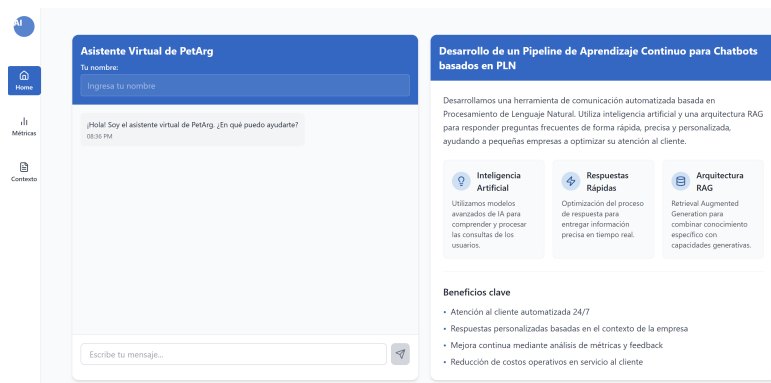


FIGURA 3.4. Frontend del chatbot.

- **Backend:** implementado en ⁵, se encarga de recibir las solicitudes del frontend para procesar las preguntas, enviar las respuestas, gestionar las métricas requeridas y actualizar los contextos en la base de datos de vectores. Gestiona el flujo completo de la conversación, registra cada interacción, clasifica temáticamente las preguntas, las redefine con el historial de mensajes previos y prepara la consulta para la lógica RAG. Además, procesa las métricas al leer las trazas de Langfuse, y analiza patrones de uso, calcula estadísticas de rendimiento y envía estos análisis al frontend para que se ilustren mediante gráficos y dashboards interactivos, lo que facilita la visualización de tendencias y áreas de mejora del sistema.
- **Guardrails:** módulo de filtrado y preprocesamiento que evalúa si una consulta es válida (según reglas predefinidas), la clasifica en categorías como "Producto", "Quejas", "Saludos" y la redefine con el contexto de los mensajes anteriores.
- **DSPy RAG:** es el corazón del sistema, responsable de orquestar la lógica principal del flujo RAG. DSPy toma la pregunta ya enriquecida desde el módulo de guardrails y ejecuta una búsqueda eficiente de contextos relevantes en LanceDB. A partir de los fragmentos recuperados, construye un prompt enriquecido que será enviado al modelo LLM para generar una respuesta coherente y precisa. Además de facilitar la integración de prompts, DSPy permite definir y estructurar toda la lógica del sistema de recuperación y generación, y proporciona un marco flexible y modular para el diseño del flujo completo.

⁵FastAPI es un framework web moderno y de alto rendimiento para construir APIs con Python, basado en las anotaciones de tipo de Python 3.6+. Se destaca por su rapidez, facilidad de uso y soporte nativo para documentación automática, lo que lo hace ideal para desarrollar backends eficientes en sistemas que integran modelos de lenguaje y flujos RAG.

- **Langfuse:** observador pasivo que registra todas las interacciones del sistema sin intervenir. Almacena información crítica como la pregunta original, la redefinida, el consumo de tokens, la respuesta final y los estados de los guardrails. También permite aplicar evaluaciones automáticas sobre la precisión de las respuestas generadas por el sistema.

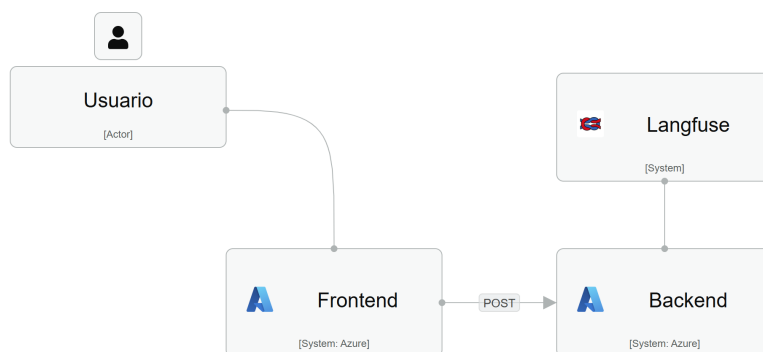


FIGURA 3.5. Diagrama general del flujo del pipeline del sistema.

3.2.1. Tecnologías suplementarias

- **Streamlit:** utilizado para construir el frontend inicial de pruebas, y permite una interacción simple con el chat, que posteriormente fue migrada a ReactJS. Streamlit es un framework de código abierto en Python para crear aplicaciones web de forma rápida y sencilla, especialmente orientado a proyectos de ciencia de datos e inteligencia artificial.

item **Lovable:** *lovable.dev* es una herramienta en línea que permite crear frontends de forma rápida y sencilla con la ayuda de inteligencia artificial. Fue utilizada para desarrollar la versión final del frontend con ReactJS, y proporcionó una experiencia de usuario más fluida y atractiva. Además, en esta plataforma se integraron las visualizaciones de las métricas generadas a partir de los datos de Langfuse.

- **Azure Container Apps:** recurso de la plataforma Azure diseñado para ejecutar aplicaciones en contenedores sin necesidad de administrar directamente la infraestructura. Fue utilizado para desplegar tanto el backend como el frontend del sistema en contenedores individuales, lo que permite escalar automáticamente según la demanda y asegura un rendimiento óptimo.

3.2.2. Funcionamiento general

El pipeline del sistema está diseñado para ofrecer una experiencia conversacional precisa, eficiente y trazable. A continuación, se detalla el flujo completo de funcionamiento desde la perspectiva funcional:

1. El usuario realiza una consulta a través del **frontend**.
2. La consulta es enviada al **backend**, donde primero pasa por un sistema de **guardrails personalizados**. En este proceso se ejecutan tres tareas clave:
 - **Filtrado:** se descartan preguntas fuera del dominio del sistema (por ejemplo, política o religión), basándose en reglas predefinidas.

- **Clasificación:** la pregunta se categoriza automáticamente según su temática (Producto, Quejas, Saludos, etc.).
 - **Redefinición:** con el historial conversacional completo, la pregunta se reformula para enriquecer su significado y contexto.
3. La pregunta redefinida es ingresada al primer módulo de DSPy, donde se utiliza para buscar en **LanceDB** los fragmentos de contexto indexados previamente que están relacionados con la pregunta en cuestión.
 4. La información obtenida se utiliza para construir un **prompt estructurado en el siguiente módulo de DSPy**. Esta es la etapa lógica más importante del pipeline, ya que aquí se encuentra el corazón del sistema RAG. En esta fase, tanto el contexto recuperado como la consulta original se integran para preparar la entrada final al modelo LLM.
 5. El prompt es enviado a un **modelo LLM** (en este caso, GPT-4o mini).
 6. La respuesta y todos los datos asociados (estado del guardrail, redefinición, tokens consumidos, clasificación, etc.) son registrados por **Langfuse**.
 7. En Langfuse, se ejecuta una **evaluación automática**, que analiza la calidad de la respuesta y asigna puntajes en una escala del 1 al 10. Las puntuaciones bajas (menores a 3) indican respuestas que solo reformulan la pregunta o evidencian insuficiencia de contexto. Los criterios de evaluación son:
 - **Precisión:** determina si la respuesta aborda directamente la pregunta del usuario o simplemente la reformula sin proporcionar información sustancial.
 - **Suficiencia de contexto:** evalúa si la respuesta demuestra que el sistema disponía de información adecuada para contestar con propiedad o si indica explícitamente la falta de contexto necesario.
 8. En paralelo con la observación de Langfuse, la respuesta es devuelta al usuario en el frontend.

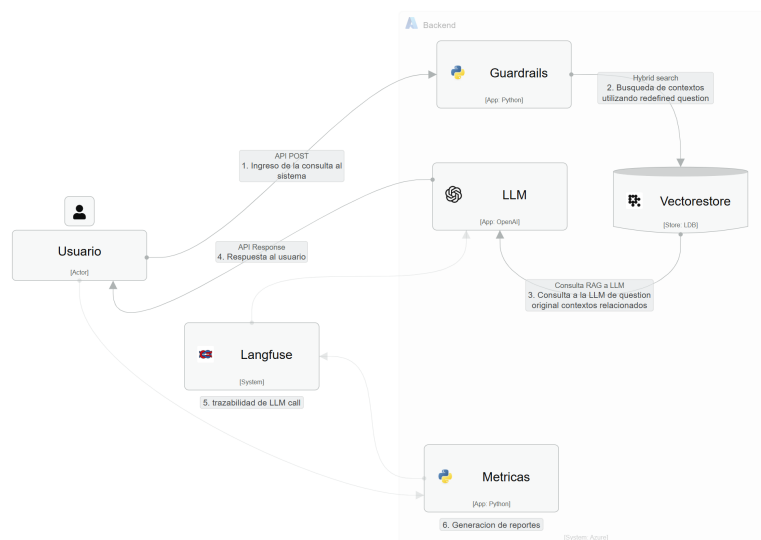


FIGURA 3.6. Diagrama de flujo de funcionamiento.

3.3. Implementación

3.3.1. Preparación de los datos de contexto

La base de conocimiento utilizada por el sistema no proviene de documentos extensos procesados en chunks, sino de una estructura de tipo clave-valor diseñada específicamente para búsquedas contextuales rápidas y precisas. Cada entrada contiene dos campos principales:

- **question:** una lista de términos, sinónimos o expresiones relacionadas que representan la intención del usuario. Este campo actúa como una clave semántica para la búsqueda.
- **relates:** una serie de frases informativas asociadas a la clave, que constituyen el contexto que será devuelto al modelo como parte del proceso RAG.

La estructura clave-valor facilita la búsqueda semántica eficiente, y permite que el sistema recupere contextos relevantes rápidamente al comparar las consultas del usuario con las claves almacenadas en la base de datos.

A continuación se muestra un ejemplo de cómo se estructuran estos datos en Python:

CÓDIGO 3.1. Ejemplo de estructura de datos clave-valor para el contexto

```
context = [
    {
        'question': [
            'queja',
            'reclamo',
            'problema'
        ],
        'relates': [
            Lamentamos que hayas tenido una mala
            experiencia. Por favor cuéntanos mas
            sobre tu situacion, Estamos aqui
            para ayudarte con cualquier
            inconveniente que hayas tenido...
        ]
    },
    {
        'question': [
            'producto',
            'características',
            'especificaciones'
        ],
        'relates': [
            Nuestro producto cuenta
            con las siguientes características,
            Las especificaciones técnicas son...
        ]
    }
]
```


Esta estructura permite una consulta eficiente, ya que las claves semánticas (como "queja", "reclamo", "producto") se utilizan para buscar y recuperar los fragmentos de contexto adecuados. Una vez que el sistema identifica las claves correspondientes a la consulta del usuario, recupera las frases informativas asociadas en el campo `relates`, que luego se integran en el flujo RAG para proporcionar respuestas contextualizadas y precisas.

Durante la interacción, el sistema utiliza un modelo de lenguaje grande (LLM) para reformular la consulta del usuario e identificar las palabras clave más relevantes. Estas palabras clave son luego utilizadas por LanceDB para realizar una comparación con el conjunto de datos almacenado en el campo `question`.

La búsqueda se realiza de manera híbrida, y combina técnicas de búsqueda semántica y vectorial. Gracias a este enfoque, el sistema no solo busca coincidencias exactas, sino que también tiene en cuenta la similitud semántica entre la consulta y las claves almacenadas. Al encontrar una coincidencia, LanceDB recupera los valores asociados desde el campo `relates`, que contienen el contexto necesario para la consulta.

Una vez obtenidos los fragmentos de contexto relevantes, estos se inyectan en el prompt del modelo LLM generador. Esto permite que el sistema genere una respuesta precisa y contextualizada, y aproveche tanto la consulta original del usuario como el contexto recuperado.

3.3.2. Sistema de reentrenamiento con historiales y evaluación automatizada de interacciones

A lo largo del desarrollo del sistema, se adoptó un enfoque alternativo al reentrenamiento tradicional de modelos de lenguaje (LLM), y se priorizó la adaptabilidad continua mediante mecanismos de evaluación automática y actualización contextual. Esta arquitectura permite que el sistema analice los resultados de las interacciones con los usuarios sin necesidad de modificar directamente los parámetros internos del modelo base.

En lugar de reentrenar el LLM, se implementó un ciclo de mejora continua, alimentado por datos conversacionales reales. Estos datos son recolectados, organizados y evaluados automáticamente mediante herramientas diseñadas para analizar el rendimiento y la calidad de las interacciones. De esta manera, se mantiene una evolución constante del sistema sin la necesidad de grandes intervenciones.

El administrador del sistema juega un papel crucial en este proceso, ya que es responsable de revisar las métricas generadas y decidir si es necesario realizar ajustes o mejoras en el sistema. A través de la interfaz de administración en el frontend, el administrador puede gestionar los contextos almacenados en la base de datos vectorial LanceDB y realizar ajustes en la configuración de los guardrails o en los datos contextuales, sin necesidad de modificar el modelo base.

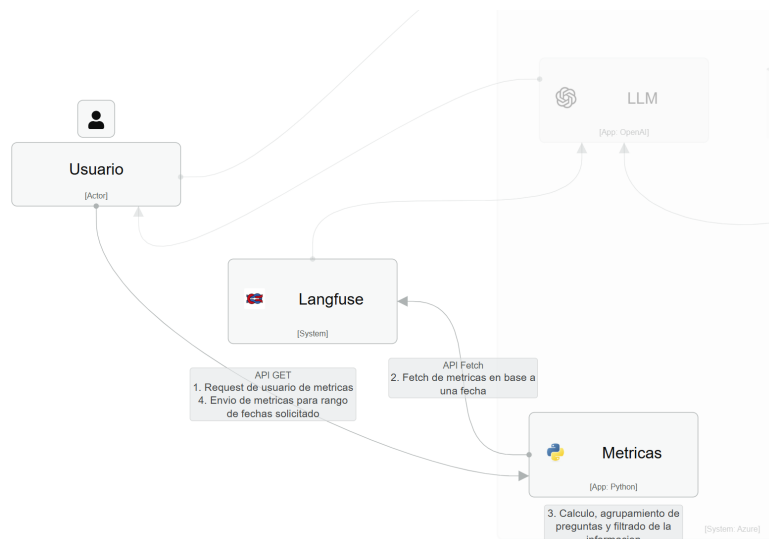


FIGURA 3.7. Diagrama de flujo de observación de métricas.

Evaluación automatizada y extracción de métricas

Cada interacción registrada en el sistema genera una “traza” que es almacenada mediante Langfuse. Estas trazas contienen no solo los mensajes de entrada y salida, sino también metainformación crucial, como el costo (tokens consumidos y dinero gastado), etiquetas temáticas (por dominio o tipo de consulta), comentarios evaluativos y puntajes de calidad (en una escala del 1 al 10).

El proceso de evaluación es automatizado y recorre todas las trazas válidas (se excluye la categoría genérica Saludo). Identifica aquellas trazas que contienen puntuaciones explícitas (‘trace.scores’). El código implementa un control de tasa (rate limit) para evitar sobrecargar el sistema y garantiza la integridad del conjunto de datos utilizado para la evaluación.

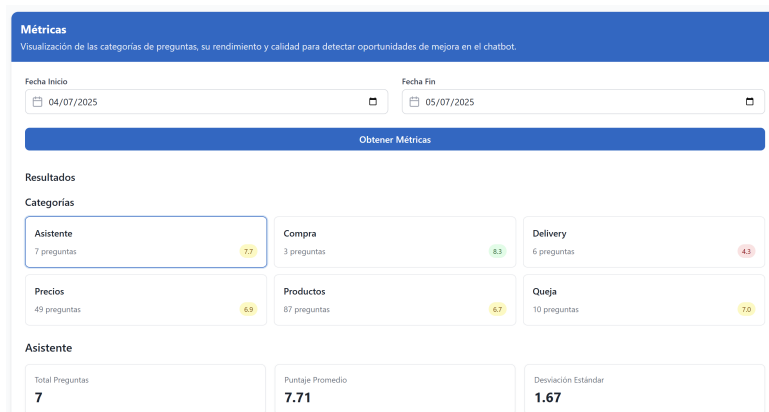


FIGURA 3.8. Métricas por categoría.

CÓDIGO 3.2. Fragmento de código para extracción y evaluación de trazas desde Langfuse

```

for trace in all_traces:
    ...
    if trace.scores and 'Saludo' not in trace.tags:
        score = trace.scores[0]
        traces_df = traces_df.append({...})

```

Procesamiento y análisis de calidad

Una vez recolectadas las trazas, se analizan mediante un pipeline estructurado que sigue estos pasos:

1. **Preprocesamiento del texto:** las preguntas son normalizadas (se convierten a minúsculas, se eliminan acentos y símbolos no relevantes) para asegurar consistencia en el análisis posterior.
2. **Asociación temática:** las preguntas se agrupan según etiquetas temáticas estandarizadas, lo que facilita la categorización y la identificación de patrones comunes en las consultas.
3. **Evaluación cuantitativa:** se asignan puntajes de calidad a cada pregunta, se procesan los valores obtenidos para calcular promedios, desviaciones estándar y clasificar las preguntas según su calidad: buenas (≥ 8), malas (<5), y aquellas con puntajes intermedios.
4. **Agrupamiento semántico:** las preguntas son transformadas en vectores de embeddings mediante técnicas de NLP, y luego se agrupan con HDBSCAN para detectar redundancias y patrones recurrentes.
5. **Síntesis por grupo:** se selecciona una pregunta representativa de cada grupo para generar un resumen conciso por cada etiqueta temática, lo que facilita la visualización de la calidad del sistema y la identificación de áreas de mejora.

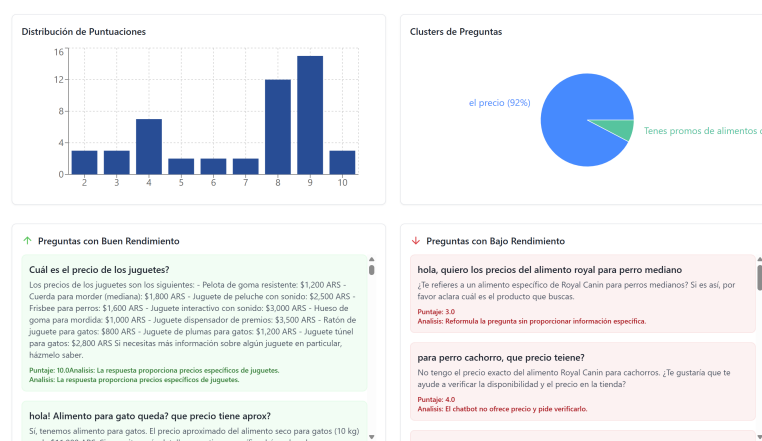


FIGURA 3.9. Ejemplo de frontend que muestra métricas para preguntas buenas y malas.

Este análisis permite generar reportes automáticos por versión del sistema. Las métricas agregadas permiten evaluar el impacto de cada iteración del sistema. Durante las diferentes versiones, se registraron las siguientes métricas:

- **Métricas de calidad de respuestas:** a lo largo de cada versión, se recopiló el puntaje promedio de calidad de las respuestas generadas. Este puntaje se calculó a partir de los puntajes de evaluación automática registrados en Langfuse, que incluyen factores como precisión, suficiencia de contexto y relevancia de la respuesta.
- **Tiempos de respuesta:** se registraron los tiempos promedio de respuesta del sistema, desde la recepción de la consulta hasta la entrega de la respuesta al usuario. Esto incluye el tiempo de procesamiento de la consulta en el backend y la generación de la respuesta por el modelo LLM.
- **Consumo de tokens y costos:** se registraron los tokens consumidos por cada interacción, así como el costo asociado, especialmente relevante en versiones donde se utilizaban servicios externos para los LLMs. Estos datos fueron clave para evaluar la viabilidad económica del sistema.
- **Interacciones clasificadas por tema:** durante cada iteración, se hizo un seguimiento de cómo se clasificaban las interacciones en diferentes temas (productos, quejas, saludos, etc.). Esto permitió evaluar si el sistema interpretaba correctamente las intenciones del usuario.
- **Tasa de satisfacción del usuario:** se implementaron encuestas o análisis de retroalimentación en el frontend, lo que permitió medir la satisfacción de los usuarios con respecto a las respuestas del chatbot.
- **Cobertura de contexto:** se evaluó la cantidad de veces que el sistema pudo proporcionar una respuesta satisfactoria gracias a la información recuperada desde la base de conocimiento. Esta métrica es útil para entender cómo la gestión y expansión de contextos impacta en la calidad del sistema.
- **Reducción de interacciones no satisfactorias:** durante las versiones posteriores, se observó la disminución de interacciones en las que el sistema no podía ofrecer una respuesta satisfactoria, lo que indica una mejora en la precisión y adaptabilidad del modelo.

Estas métricas no solo fueron fundamentales para evaluar la efectividad del sistema, sino también para dirigir futuras mejoras y ajustes. El análisis de estas métricas proporcionó información valiosa sobre qué aspectos del sistema necesitaban optimización, lo que permitió un proceso de mejora continua y garantizó que cada nueva versión fuera más eficiente y efectiva que la anterior.

3.4. Despliegue de la interfaz de pruebas

Se implementó una interfaz interactiva mediante **ReactJS**, diseñada para facilitar tanto pruebas de usuario como análisis administrativo. Esta interfaz permite:

- Realizar preguntas al chatbot y visualizar respuestas en tiempo real, lo que facilita la interacción directa con el sistema.
- Consultar métricas de evaluación automática y las categorías temáticas asignadas a cada pregunta, lo que permite un análisis detallado del rendimiento del sistema.

- Modificar o agregar fragmentos de contexto a través de un formulario de carga que ofrece la flexibilidad de ajustar la base de conocimiento en tiempo real.
- Acceder a las trazas generadas por Langfuse para cada interacción, lo que proporciona visibilidad completa sobre los datos de entrada, salida y la evaluación del sistema.

The image shows a web interface for managing context. It is divided into two main panels. The left panel, titled 'Contexto Actual', displays the current context information, including a list of topics and a detailed description. The right panel, titled 'Agregar Contexto', provides a form to add new context, with fields for 'Titulo' and 'Descripción / Contenido', and an 'Agregar' button.

Contexto Actual

INTRODUCCIÓN:

Atención, contacto, ayuda, dudas, consultas, información, disponibilidad, servicios, horarios, sugerencias

Atención:

1. Si necesitas ayuda o más información, escríbenos Estamos aquí para lo que necesites.
2. ¿Tenés dudas sobre nuestros servicios? Contactanos y te daremos todos los detalles.
3. Estamos disponibles para vos y tu mascota todos los días Consultá nuestros horarios y ubicaciones.
4. Queremos escuchar tus consultas y sugerencias. Estamos para ayudarte.

Ubicación, contacto, dirección, teléfono, email, sitio web, información de contacto

Ubicación y Contacto:

Visítanos en Av. Mascotas Felices 123, Buenos Aires, Argentina.
Contáctanos al +54 11 5555-1234.
Envíanos un correo a contacto@petargboutique.com.
Explora más en <http://petargboutique.com>.

Título de contexto: Buenos Aires de mascotas, mascotas felices

Agregar Contexto

Título

Ingresar un título descriptivo

Descripción / Contenido

Agrega información detallada que el chatbot utilizará para responder preguntas

Agregar

FIGURA 3.10. Frontend creado para la administración de contextos.

Esta interfaz está pensada como una herramienta versátil que permite validar mejoras y cambios en el sistema de manera inmediata, y asegura que las actualizaciones sean evaluadas y validadas rápidamente antes de su implementación a gran escala en el entorno de producción.

Capítulo 4

Ensayos y resultados

En este capítulo se presentan las pruebas realizadas al chatbot desarrollado, junto con los resultados obtenidos en distintos escenarios, los procesos de optimización aplicados y un análisis detallado del comportamiento observado. Asimismo, se identifican fallos relevantes y se proponen mejoras para futuras iteraciones del sistema.

Se describen los diferentes escenarios de prueba utilizados para evaluar el rendimiento del chatbot, destacando que la estructura de las pruebas fue clave para detectar áreas susceptibles de mejora y optimización. Además, se analizan los resultados obtenidos en cada escenario con el fin de evaluar la efectividad de las mejoras implementadas, así como identificar posibles limitaciones y proponer ajustes adicionales al sistema.

4.1. Evaluación de escenarios del chatbot

4.1.1. Escenario 1: Interacción básica

Se evaluó la capacidad del chatbot para mantener una conversación coherente en interacciones simples. Para ello, se formularon preguntas directamente relacionadas con el contexto actual de la versión, aunque no siempre completas ni contextualizadas en el hilo conversacional. El objetivo principal de esta prueba fue verificar el denominado *happy path* del sistema; es decir, el funcionamiento correcto de las etapas del *pipeline*, incluyendo la recepción de entrada, la captura de contextos relevantes y el análisis por parte del modelo de lenguaje para generar una respuesta adecuada.

4.1.2. Escenario 2: Consultas con múltiples pasos

En esta evaluación se realizaron consultas más complejas desde el punto de vista del hilo conversacional, donde el usuario planteó preguntas encadenadas que requerían del sistema la retención y gestión del contexto. Se analizó la capacidad del chatbot para mantener coherencia y relevancia a lo largo de múltiples interacciones. Si bien las preguntas fueron formuladas de forma directa, se buscó evaluar la habilidad del sistema para adaptarse a cambios en la temática sin perder el hilo de la conversación ni mezclar las respuestas con temas tratados previamente.

4.1.3. Escenario 3: Consultas fuera de contexto

En este escenario se examinó la capacidad del chatbot para gestionar preguntas ambiguas o fuera de contexto, así como su habilidad para solicitar mayor información o derivar la consulta a un humano. Cabe destacar que esta función se encuentra en etapa de pruebas, ya que el prototipo no cuenta con una conexión real a un sistema CRM para realizar la transferencia de la conversación. El objetivo fue evaluar la habilidad del sistema para reconocer sus propias limitaciones y ofrecer respuestas apropiadas incluso ante la falta de información contextual.

4.2. Resultados de las pruebas

En este apartado se presentan los resultados obtenidos en cada uno de los escenarios de prueba descritos anteriormente para cada una de las 4 versiones del chatbot. Además, se discuten los hallazgos más relevantes y se proponen recomendaciones para futuras mejoras.

4.2.1. Pruebas para la version 0

Pruebas de interacción básica

Pruebas de consultas con múltiples pasos

Pruebas de consultas fuera de contexto

Conclusiones de la version 0

4.2.2. Pruebas para la version 1

Pruebas de interacción básica

Pruebas de consultas con múltiples pasos

Pruebas de consultas fuera de contexto

Conclusiones de la version 1

4.2.3. Pruebas para la version 2

Pruebas de interacción básica

Pruebas de consultas con múltiples pasos

Pruebas de consultas fuera de contexto

Conclusiones de la version 2

4.2.4. Pruebas para la version 3

Pruebas de interacción básica

Pruebas de consultas con múltiples pasos

Pruebas de consultas fuera de contexto

Conclusiones de la version 3

4.3. Mejoras y optimizaciones propuestas

En esta sección se presentan las mejoras y optimizaciones propuestas para el chatbot, basadas en los resultados obtenidos en las pruebas realizadas. Estas recomendaciones están orientadas a abordar los problemas identificados y mejorar la experiencia del usuario, así como la eficiencia del sistema en futuras iteraciones.

4.3.1. Mejoras en la gestión del contexto

4.3.2. Optimización del modelo de lenguaje

4.3.3. Integración con sistemas externos

4.3.4. Mejoras en la interfaz de usuario

Bibliografía

- [1] IBM. *Natural Language Processing (PLN)*.
<https://www.ibm.com/mx-es/topics/natural-language-processing>.
Accedido el 17 de septiembre de 2024. 2023.
- [2] Cobus Greyling. *RAG vs Fine-Tuning*.
<https://cobusgreyling.medium.com/rag-fine-tuning-e541512e9601>.
Accedido el 20 de septiembre de 2024. 2023.